P1 - 2.0
P2 2.5
P3 1.5
P4 2.5
9.5

Thôm Trường Giang
MSSV: N15 DECE080
Lớp: E25 CQCE02-IV.

Problem 1: Singly linked list - In place Reversal & Pointer tracing.

. Task 1: Write a function/algorithm in pseudocode to reverse the singly linked-list including three pointers (prev, curr, next) after each iteration until the list is fully reversed.

Note: Do not use algorithm library.

Problem 2: Circular linked-list - Insert at head.

Given a singly circular linked list: head -> [10] -> [20] -> [30] -> (points back to head)

. Task 1: Write pseudocode for the function Insert Head (head, new value) to insert a new node containing new value at the front of the circular linked list while maintaining its circular structure.

Quang Tâm

Scanned with CamScanner

• Task 2 Trace the execution steps (via text description or pointer diagrams) when inserting new value = 5 into given.

Problem 3 Doubly linked - list - Insert B4 Target & Edge Cases

Given a doubly linked list containing integers.

• Task 1: write pseudocode for InsertBefore (head, target, new value) to insert a new node containing new value immediately B4 the first occurrence of target.

• Task 2: list the exact order of pointers assignments required and briefly justify why this specific sequence avoids memory leaks or broken links.

• Task 3: Explicitly handle the following 2 edge cases in code / pseudocode.

1. target is located at the head node
2. Target does not exist in the list.

Problem 4: Sorted linked - list - Inorder Insertion.

Quang Tâm

Scanned with CamScanner

Given a singly linked list sorted in ascending order: head → [2] -> [5] -> [8] → null

Task 1: Write the Function SortedInsert (head, val) to insert val into the list such that the ascending order is preserved.

Task 2: Construct a step by step dace table detailing pointer iterations and condition evaluations when inserting val = 6 into the list.

Note: Do not use Algorithms library.

Solution

Problem 1

Task 1: Pseudo code:

```
Function ReverseList (head):
    prev = NULL
    curr = head
    next = NULL
While (curr is not NULL):
    nex = curr.next
    curr.nex = prev
```


Quang Tâm

Scanned with CamScanner

prev = curr

curr = next

head = prev

return head

0.75 Task 2: Trace table:

Given initial list: ~~Ked~~ head -> [10] -> [25] -> [40] - [15] -> NULL

Linked Strict ?

State (End of Iteration)	prev	curr	next
Initialization	NULL	[10]	NULL
Iteration 1	[10]	[25]	[25]
Iteration 2	[25]	[40]	[40]
Iteration 3	[40]	[15]	[15]
Iteration 4	[15]	NULL	NULL

Final result:

After Iteration 4, the loop ends and the ~~Ked~~ head is assigned by prev which is ~~node~~ [15]

2.5 Problem 2:

1.25 Task 1: Pseudocode

Function InsertHead (head, new value):

new node = create Node (new value)

if head is NULL:

new Node. next = new Node.

Quang Tâm

Scanned with CamScanner

Thôm Thường Giang - M25DECB080

~~Else~~ head = newNode
return head
newNode.next = head
temp = head
while temp.next != head:
temp = temp.next
temp.next = newNode
head = newNode
return head

1.25 Task 2: Execution steps (newValue = 5)
Given list: head -> [10] -> [20] -> [30] -> (points back to head)
1. Create newNode with value 5
2. Set newNode.next = head (5 points to [10])
3. Initialize temp = head ([10]) and traverse until temp.next == head. temp moves from [10] -> [20] -> [30]. temp stops at [30]

Quang Tâm
Scanned with CamScanner

4. Set temp.next = newNode([10] now points to 5 instead of [60]).

5. Update head = newNode(head now points to 5)

Final list: Head = newNode(head -> [5] -> [10] -> [20] -> [30] -> points back to head)

1,5 Problem 3:

0,75 Task 1 & 3:

Function InsertBefore(head, target, newValue)

if head is NULL

return head.

if (head.value == target)

newNode = CreateNode(value)

newNode.next = head.

newNode.prev = NULL.

head.prev = newNode

head = newNode

return head.

Sai chỗ kiểm tra vòng lặp, ko tạo node trước khi gán

while (curr != null)

if curr = head.

curr = curr.next (while curr != null && curr.value != target)

if curr is NULL:

return head

Quang Tâm

Scanned with CamScanner

newNode = CreateNode(value).
newNode.next = curr.
newNode.prev = curr.prev.
if(curr.prev != NULL)
{curr.prev.next = newNode;}
curr.prev = newNode
return head.

0,75 Task 2:
newNode.next ← curr
newNode.prev ← curr.prev
If curr.prev ≠ NULL:
curr.prev.next ← newNode
curr.prev ← newNode.
This order preserves the original links before changing them, prevents broken links and losing access to nodes.

2,5 Problem 4:

1,25 Task 1:
Function SortedInsert(head, val):
newNode ← CreateNode(val)
If(head = NULL or val < head.value)
newNode.next ← head.
head ← newNode

Quang Tâm
Scanned with CamScanner

Return head.
curr ← head.
While (curr.next ≠ NULL and curr.next.value < val:)
newNode.next ← curr.next
curr.next ← newNode
Return head.

Task 2: trace for val

Step	curr	curr.next	Condition	Action
Initial	2	5	6 < 2 → False	Not inserted here
1	2	5	5 < 6 → True	curr = 5
2	5	8	8 < 6 → False	Stop
Insert	5	8	-	Insert

Final list:

head → [2] → [5] → [6] → [8] → NULL